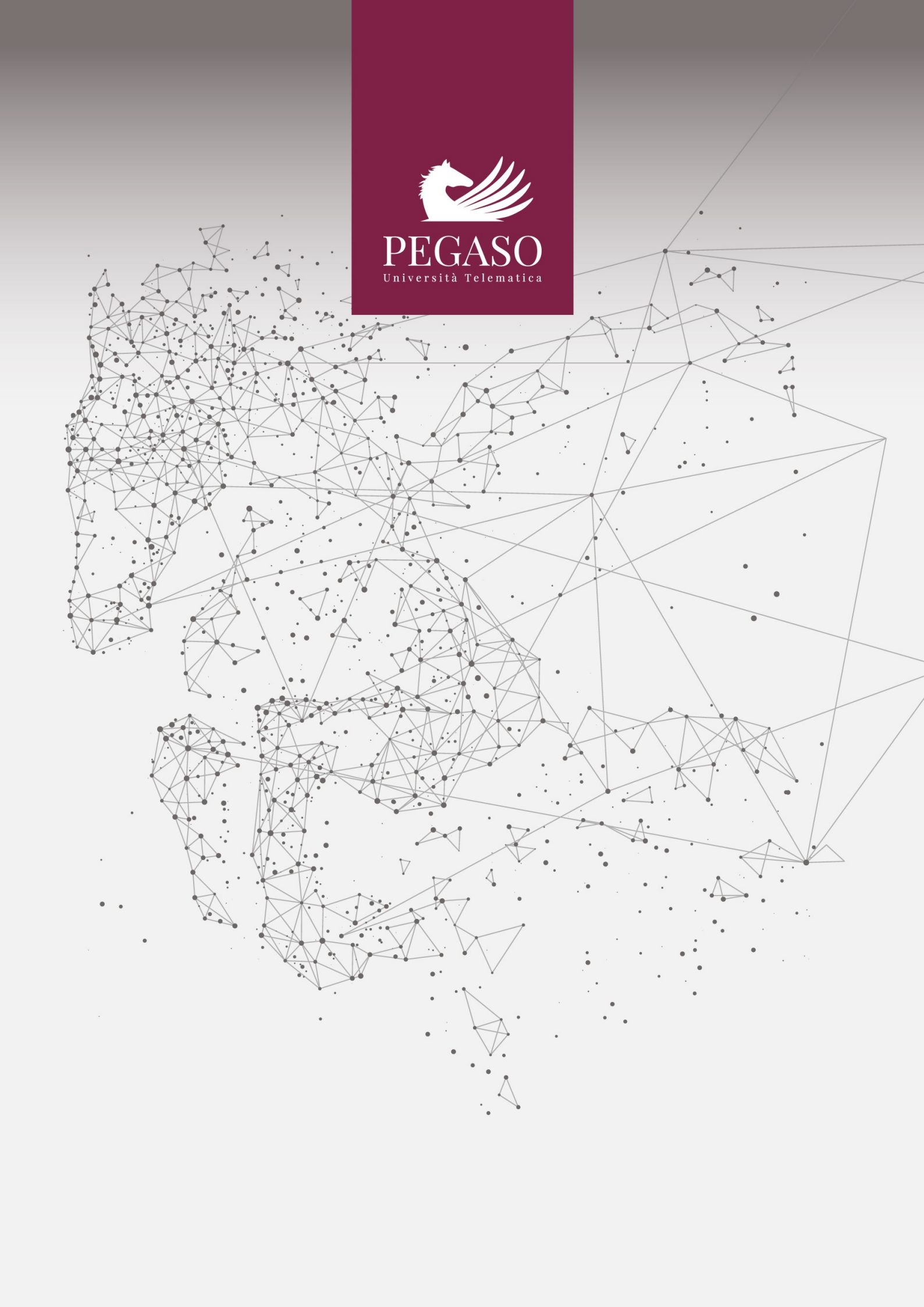

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ALGORITMO DI SCHEDULING 3
2. COMPRESSIONE DI HUFFMAN 5
3. IMPLEMENTAZIONE 12
BIBLIOGRAFIA 14

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Algoritmo di scheduling

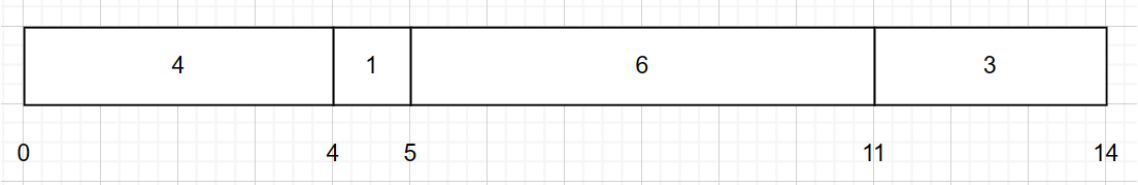
Supponiamo di avere un processore e n job da eseguire su di esso, ognuno caratterizzato da un tempo di esecuzione $t[i]$ noto a priori.

Occorre trovare una sequenza di esecuzione (permutazione) che minimizzi il tempo di completamento medio.

Dato un vettore $A[1 \dots n]$ contenente una permutazione di $\{1, \dots n\}$, il tempo di completamento dell'h-esimo job nella permutazione è:

$$T_A(h) = \sum_{i=1}^{h} t[A[i]]$$

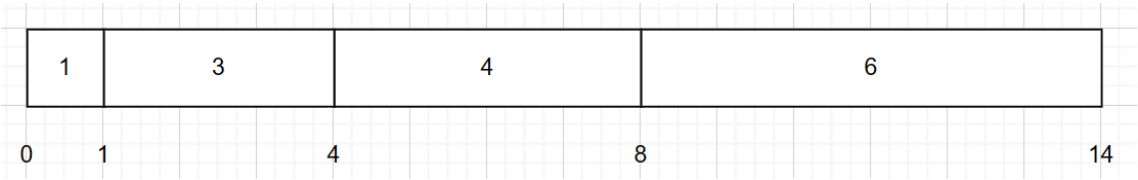
Consideriamo i seguenti task da svolgere ed eseguiamoli esattamente nell'ordine per cui i task si sono presentati: $t[i] = \{4, 1, 6, 3\}$



In questo caso il tempo di completamento medio è:

$$(4 + 5 + 11 + 14)/4 = 34/4 = 8.5$$

Adottiamo la strategia dello shortest job first:



In questo caso il tempo di completamento medio è:

$$(1 + 4 + 8 + 14)/4 = 27/4 = 6.75$$

Dimostriamo la correttezza della soluzione, e cioè che la soluzione ottima è quella che pone i job con il minor tempo di fine in sequenza (dal più corto al più lungo).

Sia A una soluzione ottima di un problema con n job, in cui il job con minor tempo di fine m si trova in prima posizione; la permutazione dei seguenti $n - 1$ job in A è una soluzione ottima al sottoproblema in cui il job m non viene considerato.

Ragioniamo nel seguente modo: supponiamo di avere una soluzione ottima A in cui però il job m con il minor tempo di fine non è posto in prima posizione ma si trovi altrove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Effettuiamo lo scambio del job in posizione 1 con il job in posizione m ottenendo dunque una nuova permutazione A'.

Vogliamo calcolare il tempo medio di completamento di A' e vogliamo dimostrare che questo è minore o uguale a quello di A.

Quindi la situazione è la seguente:

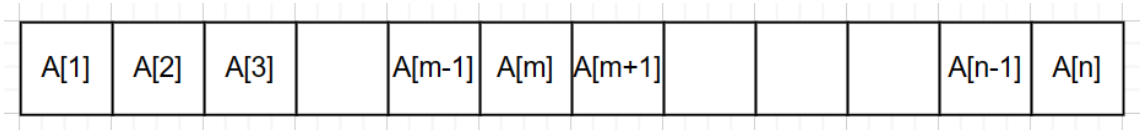


Figura 1 - Permutazione A

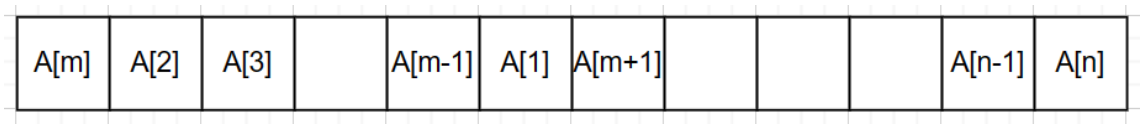


Figura 2 - Permutazione A'

Se considero i job della permutazione A' da $A[m+1]$ ad $A[n]$, in realtà non cambia nulla a livello di tempo di completamento perché di fatto ciò che è successo è che si sono scambiati di posto "davanti" a loro alcuni job, pertanto il loro tempo di completamento non sta cambiando (è come se ci trovassimo in coda e davanti a noi delle persone si scambiano di posto: a noi che ci troviamo in coda non cambia nulla).

Il fatto che $A[m]$, il job con tempo di fine più corto (per ipotesi), sia ora in prima posizione, "potenzialmente" fa sì che i job dal primo ($A[m]$) fino alla posizione occupata da $A[m-1]$ possano avere migliorato il proprio tempo di completamento rispetto allo scenario della permutazione A. In sostanza, essendo $A[m]$ con un tempo di completamento minore di $A[1]$ rende tutta la sequenza dei primi $m-1$ job di A' potenzialmente con un tempo di completamento minore rispetto ai primi $m-1$ job della sequenza A.

Se consideriamo ora i primi m job in A ed A' (quindi aggiungiamo la posizione m all'analisi appena fatta), abbiamo che complessivamente il tempo di completamento relativamente a questi primi m job è identico o è minore per A' alla luce del potenziale miglioramento di cui abbiamo discusso in precedenza.

Quindi, riepilogando, A' potrebbe avere un tempo di completamento minore se consideriamo i primi m job rispetto ad A, mentre dal job $m+1$ fino ad m il tempo di completamento è identico.

Ma se abbiamo detto che A è ottima per ipotesi, allora non è possibile che A' abbia un tempo di completamento inferiore, pertanto anche A' è una soluzione ottima.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Compressione di Huffman

Il problema della compressione riguarda la rappresentazione dei dati in modo efficiente, in particolare consiste nell'impiegare il numero minore di bit per la rappresentazione.

Una possibile tecnica di compressione è quella della codifica di caratteri tramite funzione di codifica $f : f(c) = x$

- c è un possibile carattere preso da un alfabeto Σ
- x è una rappresentazione binaria
- c è rappresentato da x

In un **codice a prefisso**, nessun codice è prefisso di un altro codice (condizione necessaria per la decodifica).

Se ad esempio la codifica è la seguente:

- "a" → 0
- "b" → 10
- "c" → 11

Riesco ad eseguire sempre la decodifica.

Se ho ad esempio la seguente codifica:

- "a" → 0
- "b" → 1
- "c" → 11

Ho una ambiguità e non so eseguire sempre la decodifica.

David Huffman, introduce un algoritmo ottimo per costruire codici prefissi; tale algoritmo è oggi utilizzato come complemento di altri metodi di compressione (e.g. in pkzip, zip, winrar).

Principio del **codice di Huffman**:

- minimizzare la lunghezza dei caratteri che compaiono più frequentemente
- assegnare ai caratteri con la frequenza minore i codici corrispondenti ai percorsi più lunghi all'interno di un albero di decodifica
- utilizzo di una rappresentazione ad albero per la decodifica con il seguente algoritmo:

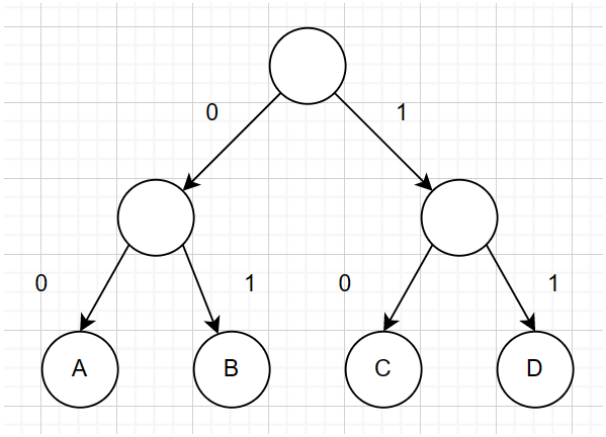
```
Algoritmo di decodifica
parti dalla radice
while file non è finito
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
leggi un bit
if bit è zero
     vai a sinistra
else
     vai a destra
if nodo foglia
     stampa il carattere
     torna alla radice
```

In questo albero di decodifica:



Abbiamo che:

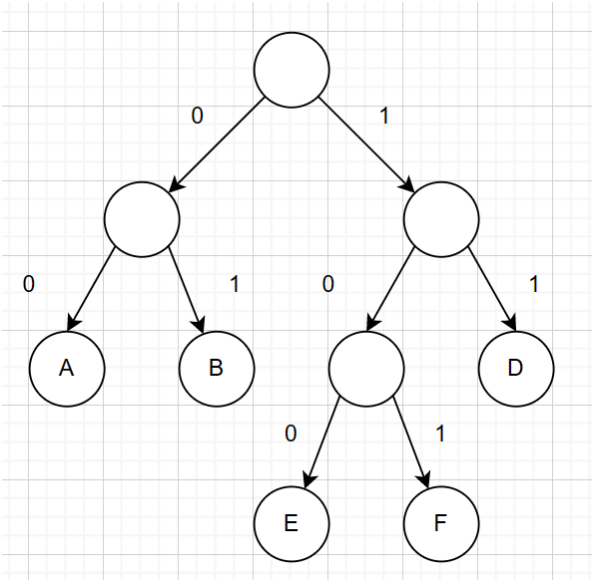
A → 00

B → 01

C → 10

D → 11

Supponiamo di avere il seguente albero:



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Abbiamo che:

A → 00

B → 01

D → 11

E → 100

F → 101

Ovviamente potrei costruire l'albero in modi differenti; la scelta ottima è quella di andare a scegliere una profondità inferiore (e quindi meno bit) per i caratteri che appaiono più frequentemente.

Consideriamo dunque un file F composto da caratteri nell'alfabeto Σ

Sia T un albero che rappresenta la codifica; per ogni $c \in \Sigma$, sia $d_T(c)$ la profondità della foglia che rappresenta c.

Il codice per c richiederà allora $d_T(c)$ bit

Se $f[c]$ è il numero di occorrenze di c in F, allora la dimensione della codifica è

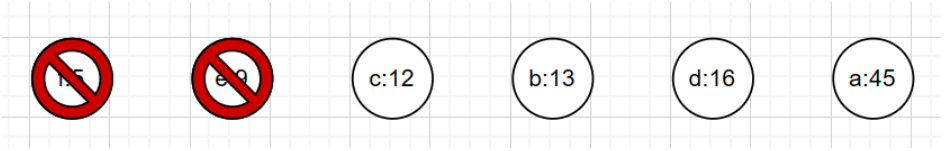
$$C(F,T) = \sum_{c \in \Sigma} f[c] \cdot d_T(c)$$

Il funzionamento dell'algoritmo è il seguente:

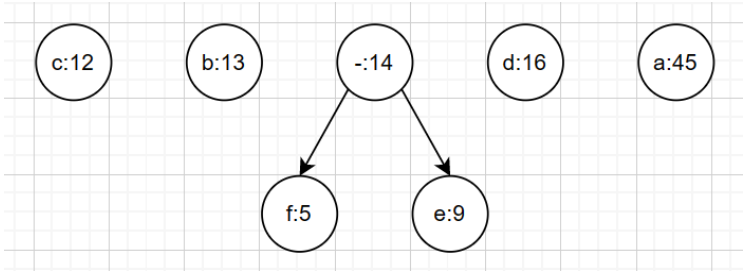
- Costruire un nodo foglia per ogni carattere, etichettato con la propria frequenza
- Ordino i nodi da quello con frequenza minore al maggiore



- Rimuovere i due nodi con frequenze minori f_x, f_y

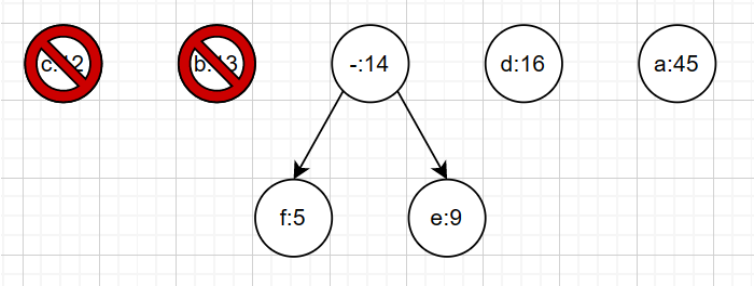


- Creare un nodo padre con etichetta "-" e frequenza f_x+f_y
- Collegare i due nodi rimossi con il nuovo nodo
- Aggiungere il nodo così creato all'insieme

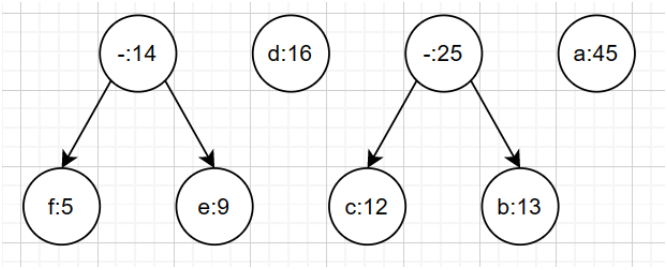


Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

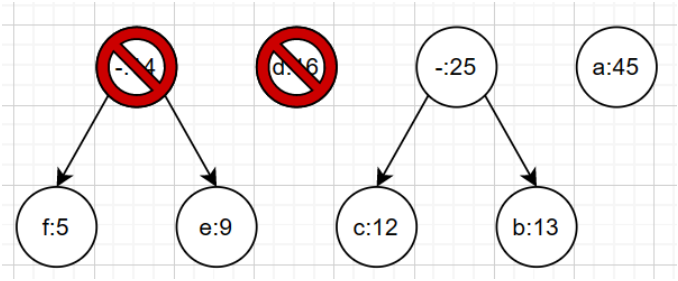
- Rimuovere i due nodi con frequenze minori f_x, f_y



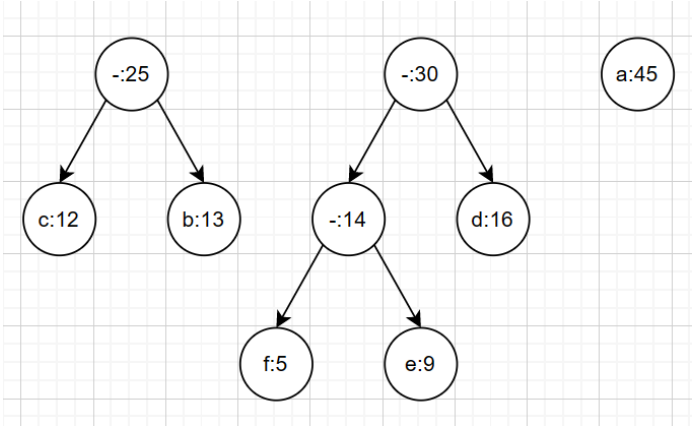
- Creare un nodo padre con etichetta "-" e frequenza f_x+f_y
- Collegare i due nodi rimossi con il nuovo nodo
- Aggiungere il nodo così creato all'insieme



- Rimuovere i due nodi con frequenze minori f_x, f_y



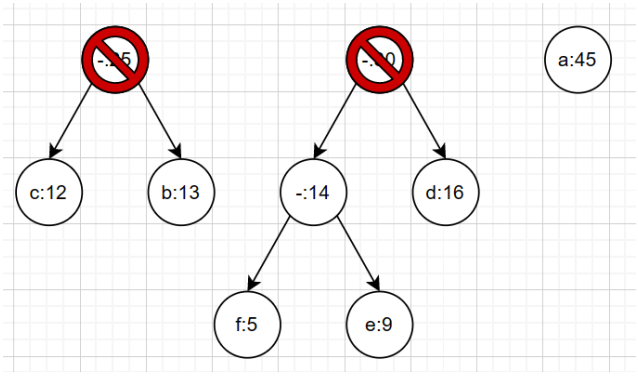
- Creare un nodo padre con etichetta "-" e frequenza f_x+f_y
- Collegare i due nodi rimossi con il nuovo nodo
- Aggiungere il nodo così creato all'insieme



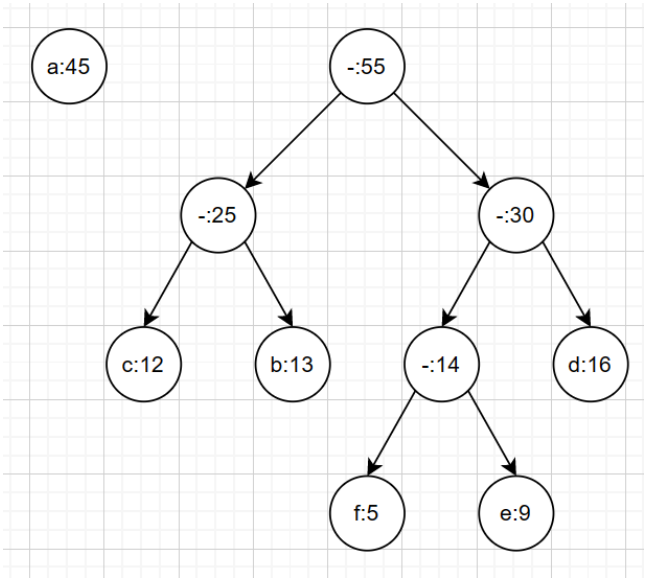
PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

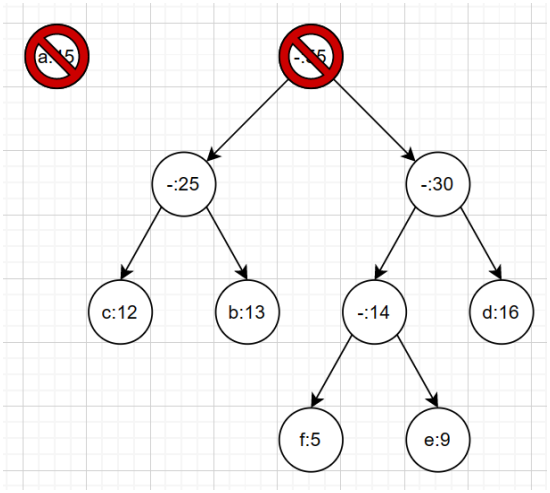
- Rimuovere i due nodi con frequenze minori f_x, f_y



- Creare un nodo padre con etichetta "-" e frequenza f_x+f_y
- Collegare i due nodi rimossi con il nuovo nodo
- Aggiungere il nodo così creato all'insieme

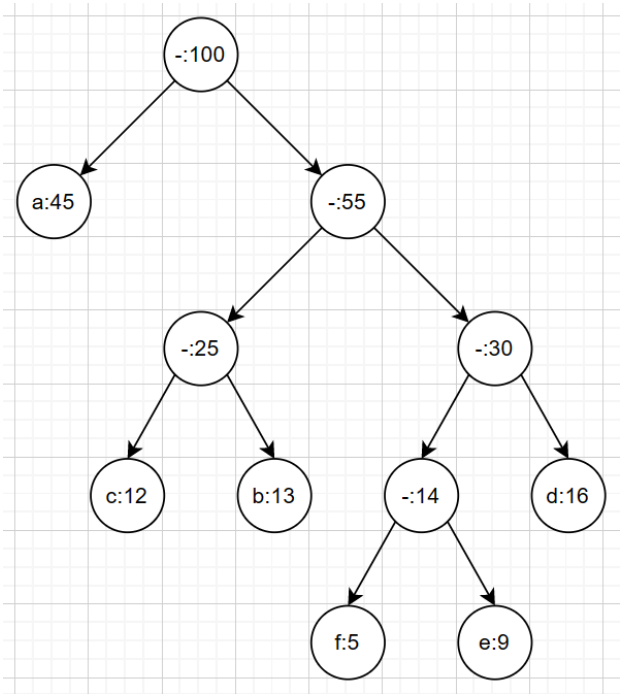


- Rimuovere i due nodi con frequenze minori f_x, f_y

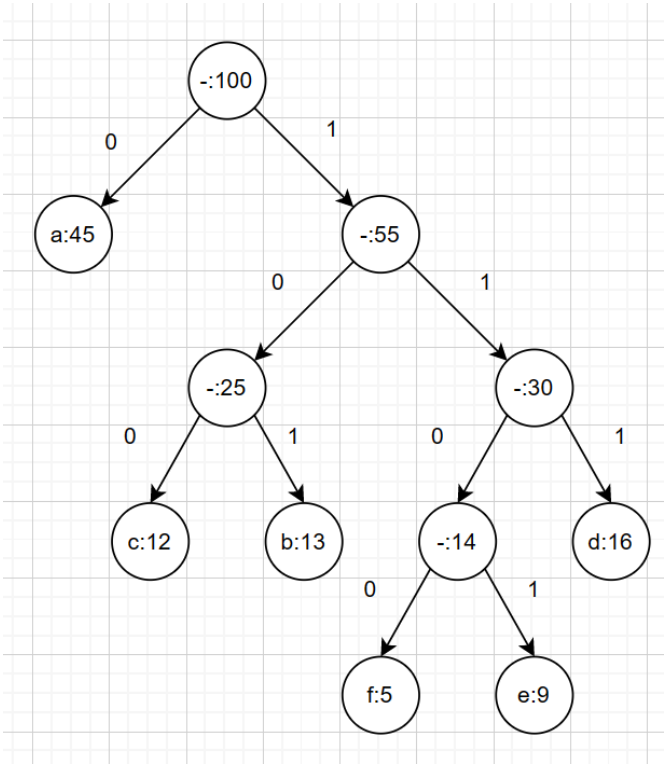


Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- Creare un nodo padre con etichetta "-" e frequenza f_x+f_y
- Collegare i due nodi rimossi con il nuovo nodo
- Aggiungere il nodo così creato all'insieme



Inseriamo i valori 0 ed 1 negli archi:



Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La tabella di codifica risultante sarà:

lettera	codifica
a	0
b	101
c	100
d	111
e	1101
f	1100

Ecco l'algoritmo che realizza l'albero di Huffman:

```
Tree huffman(int[] c, int[] f, int n)
PriorityQueue Q = MinPriorityQueue()
for i = 1 to n
     Q.insert(Tree(c[i], f[i]), f[i])
for i = 1 to n – 1
     z1 = Q.deleteMin()
     z2 = Q.deleteMin()
     z = Tree(nil, z1.f + z2.f)
     z.left = z1
     z.right = z2
     Q.insert(z, z.f)
return Q.deleteMin()
```

La complessità dell'algoritmo è $O(n \log n)$

La proprietà della scelta greedy consiste nello scegliere i due elementi con la frequenza più bassa: questo conduce sempre ad una soluzione ottimale.

È possibile dimostrare che questa scelta greedy sia effettivamente la soluzione ottima.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Implementazione

Di seguito il codice in Python per implementare alcuni algoritmi greedy.

Algoritmo di Huffman:

```
from heapq import heappush, heappop, heapify

def huffman_encode(freq):
    heap = [[wt, [sym, ""]] for sym, wt in freq.items()]
    heapify(heap)
    while len(heap) > 1:
        lo = heappop(heap)
        hi = heappop(heap)
        for pair in lo[1:]:
            pair[1] = '0' + pair[1]
        for pair in hi[1:]:
            pair[1] = '1' + pair[1]
        heappush(heap, [lo[0] + hi[0]] + lo[1:] + hi[1:])
    return sorted(heappop(heap)[1:], key=lambda p: (len(p[-1]), p))

freq = {'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}
huffman_encoded = huffman_encode(freq)
for p in huffman_encoded:
    print(f"{p[0]}: {p[1]}")
```

Algoritmo di money_change:

```
def coin_change_greedy(amount, coins):
    coins.sort(reverse=True)
    change = []
    for coin in coins:
        while amount >= coin:
            amount -= coin
            change.append(coin)
    return change

amount = 48
coins = [1, 5, 10, 25]
change = coin_change_greedy(amount, coins)
print(change)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Algoritmo di scheduling:

```
def activity_selection(start, finish):
    n = len(finish)
    activities = []
    i = 0
    activities.append(i)
    for j in range(1, n):
        if start[j] >= finish[i]:
            activities.append(j)
            i = j
    return activities

start = [1, 3, 0, 5, 8, 5]
finish = [2, 4, 6, 7, 9, 9]
selected_activities = activity_selection(start, finish)
     print(selected_activities)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Alan Bertossi, Alberto Motresor: Algoritmi e strutture di dati, Città Studi Edizioni, terza edizione;
- C. Demetrescu, I. Finocchi, G. F. Italiano: Algoritmi e strutture dati, McGraw-Hill, seconda edizione;
- Crescenzi, Gambosi, Grossi: Strutture di Dati e Algoritmi, Pearson/Addison-Wesley;
- Sedgewick: Algoritmi in C, Pearson, 2015;
- Cormen Leiserson Rivest Stein-Introduzione Agli Algoritmi E Strutture Dati-Prima Edizione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).